Rapport sur les algorithmes de génération d’entités nommées

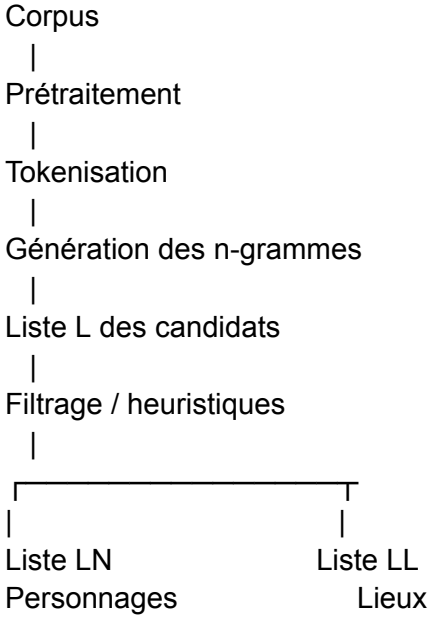
1. Principe général de l’algorithme

L'objectif de notre travail est d'extraire automatiquement des entités nommées à partir du corpus français d'Isaac Asimov fourni dans le cadre du projet. Nous cherchons principalement à identifier les **personnages** et les **lieux** présents dans les trois romans du cycle de *Fondation*.

Notre approche repose sur plusieurs étapes successives : prétraitement du corpus, tokenisation, génération de n-grammes, création d'une liste générale de candidats, puis filtrage de ces candidats afin d'obtenir une liste de personnes et une liste de lieux.

L'idée générale est de ne pas chercher directement les personnages et les lieux. Nous construisons d'abord une liste large de candidats, puis nous appliquons progressivement différentes règles permettant de réduire le bruit.

L'architecture générale de notre algorithme est la suivante :



2. Prétraitement et tokenisation du corpus

Avant de rechercher les entités nommées, le corpus doit être préparé afin de faciliter son traitement automatique.

Le corpus étant fourni sous forme de fichiers texte UTF-8, nous conservons cet encodage afin de ne pas perdre les caractères accentués du français.Une première étape consiste à

parcourir le texte et à le transformer en une séquence de tokens. Un token correspond généralement à un mot ou à un élément de ponctuation.

Par exemple : Hari Seldon arrive sur Trantor. peut être transformé en :Hari Seldon

La ponctuation est traitée séparément afin de ne pas fusionner des mots différents.

Nous devons cependant faire attention au nettoyage du corpus. Une pression trop importante de la ponctuation pourrait faire disparaître des informations utiles pour déterminer les limites des phrases ou des groupes de mots.

Le corpus présente également une particularité : seules les pages impaires ont été conservées. Certains mots ou paragraphes peuvent donc être incomplets. Notre algorithme doit fonctionner malgré ces imperfections.

3. Génération des n-grammes

Après la tokenisation, nous générons des n-grammes. Un n-gramme est une séquence de n tokens consécutifs. Nous utilisons principalement des unigrammes, bigrammes et trigrammes.

Pour la phrase : Hari Seldon arrive sur Trantor, les unigrammes sont : Hari, Les bigrammes sont : Hari Seldon

Les trigrammes sont : Hari Seldon arrive

Les n-grammes sont utiles car certaines entités nommées sont composées de plusieurs mots. Une approche utilisant uniquement des mots isolés pourrait ainsi séparer une entité telle que `Hari Seldon` en deux éléments différents.

Nous pouvons également compter la fréquence de chaque n-gramme. Une entité apparaissant régulièrement dans le corpus constitue un candidat plus intéressant qu'une séquence apparaissant une seule fois.

4. Tâche 1 : génération de la liste L

La première tâche consiste à générer une liste L contenant les candidats susceptibles d'être des entités nommées.

L'algorithme parcourt les tokens du corpus et sélectionne notamment les mots commençant par une majuscule. Cette caractéristique est intéressante car les noms de personnes et de lieux commencent généralement par une majuscule.

Cependant, cette règle produit beaucoup de faux positifs. Par exemple, le premier mot d'une phrase commence également souvent par une majuscule alors qu'il ne s'agit pas nécessairement d'un nom propre.

Pour cette raison, la liste L est volontairement une liste de candidats et non une liste définitive d'entités. Les candidats peuvent également être construits à partir des n-grammes. Lorsqu'une séquence de plusieurs mots commence par des majuscules, elle peut être conservée comme candidat composé. Exemple : Hari Seldon est conservé comme un candidat possible. Nous pouvons également associer à chaque candidat sa fréquence dans le corpus : candidat fréquence

5. Tâche 2 : génération de la liste LN des personnages

La deuxième tâche consiste à transformer la liste générale L en une liste LN contenant les candidats correspondant à des personnes ou personnages. Pour cela, plusieurs heuristiques sont combinées.

5.1. Filtrage par l'antidictionnaire

Nous utilisons l'antidictionnaire français fourni avec le projet. Celui-ci contient des mots fonctionnels et des mots très fréquents comme : le, la

Ces éléments ne sont généralement pas des noms de personnages. Ils peuvent donc être supprimés des candidats. Cette étape permet de réduire considérablement le bruit.

5.2. Utilisation des majuscules

Un autre indice utilisé est la présence de majuscules. Les noms propres ont généralement une initiale majuscule. Lorsqu'un candidat possède cette caractéristique et qu'il apparaît plusieurs fois dans le corpus, il peut être conservé comme candidat de personnage. Pour les noms composés, nous pouvons également rechercher des séquences de deux ou trois mots commençant par une majuscule.

5.3. Informations morphosyntaxiques

Une autre possibilité consiste à utiliser un outil d'annotation morphosyntaxique tel que **Freeling**. L'outil permet d'obtenir des informations grammaticales sur les tokens et peut notamment aider à identifier les noms propres. Ces informations sont utilisées comme un indice supplémentaire plutôt que comme une règle unique, car l'annotation automatique peut également produire des erreurs.

5.4. Fréquence

La fréquence d'apparition constitue également une information utile. Un candidat qui apparaît plusieurs fois dans différents passages du corpus est plus intéressant qu'un candidat qui apparaît uniquement une fois. La fréquence ne permet cependant pas à elle seule de déterminer qu'un élément est un personnage. Elle est donc combinée aux autres heuristiques.

6. Tâche 3 : génération de la liste LL des lieux

La troisième tâche consiste à identifier les lieux à partir de la liste des candidats. Nous appliquons des principes similaires à ceux utilisés pour les personnages : majuscules, n-grammes, fréquence et informations linguistiques. Le contexte des candidats peut également être exploité. Par exemple, lorsqu'un nom apparaît dans une structure telle que : sur Trantor. les mots qui l'entourent fournissent des indices permettant de considérer `Trantor` comme un lieu.

Les lieux peuvent être de différentes natures : villes, planètes, organisations ou autres entités géographiques et institutionnelles présentes dans le récit. La liste `LL` est donc obtenue en appliquant progressivement des règles de filtrage aux candidats de `L`.

7. Résumé de l'algorithme

Notre méthode peut être résumée en plusieurs étapes :

1. Lecture du corpus UTF-8
   ↓
2. Prétraitement du texte
   ↓
3. Tokenisation
   ↓
4. Génération des 1-, 2- et 3-grammes
   ↓
5. Calcul des fréquences
   ↓
6. Génération de L
   ↓
7. Suppression du bruit avec l'antidictionnaire
   ↓
8. Identification des candidats de personnes
   ↓
9. Construction de LN
   ↓
10. Identification des candidats de lieux
   ↓
11. Construction de LL

L'intérêt de cette approche est de combiner plusieurs indices plutôt que de dépendre d'une seule règle. Les majuscules permettent de trouver des candidats, les n-grammes permettent de détecter les noms composés, l'antidictionnaire permet de réduire le bruit et les informations morphosyntaxiques peuvent améliorer la classification.

Conclusion

L'algorithme développé repose donc sur une stratégie de **génération puis filtrage**. Dans un premier temps, nous privilégions le rappel en construisant une liste `L` assez large. Dans un second temps, nous appliquons différentes heuristiques pour obtenir des listes plus

spécialisées : LN pour les personnages et LL pour les lieux. Cette approche permet de traiter automatiquement un corpus littéraire et de transformer le texte brut en informations structurées. Les listes obtenues constituent ensuite une base pour la génération automatique du réseau de personnages.